

**ECOLE NATIONALE DES INGENIEURS DES TRAVAUX AGRICOLES
DE BORDEAUX**

DEPARTEMENT ENTREPRISE ET SYSTEMES

UNITE DE FORMATION INFORMATIQUE ET GENIE DES EQUIPEMENTS

~o~O~o~

INITIATION AU LANGAGE SQL

Notes de cours

Mai 2003

Monique BORIES, Jérôme STEFFE, François THIBERVILLE

SOMMAIRE

A - GENERALITES	3
B - REQUETES DE SELECTION SUR UNE TABLE	4
1 - L'INSTRUCTION SELECT	4
2 - CLAUSE DISTINCT	4
3 - CLAUSE WHERE	4
4 - UTILISATION DES ATTRIBUTS BETWEEN, IN, LIKE	5
5 - UTILISATION D'EXPRESSIONS	6
6 - UTILISATION DE FONCTIONS	6
A) FONCTIONS MATHÉMATIQUES ET STATISTIQUES	6
B) FONCTIONS DE DATE	7
C) FONCTIONS RELATIVES AUX CARACTÈRES	7
7 - CLASSEMENT DES DONNÉES	8
8 - REGROUPEMENT D'ENREGISTREMENTS	8
A) - CLAUSE GROUP BY	8
B) - CLAUSE HAVING	8
C - REQUETE DE SELECTION MULTITABLES - LES JOINTURES	10
1 - JOINTURE DE 2 TABLES	10
- JOINTURE DE PLUS DE 2 TABLES	12
3 - JOINTURE D'UNE TABLE AVEC ELLE-MÊME (AUTO-JOINTURE)	12
4 - JOINTURE EXTERNE	13
D - REQUETES IMBRIQUEES	14
1 - REQUÊTES IMBRIQUÉES RENVOYANT UNE VALEUR UNIQUE	14
2 - REQUÊTES IMBRIQUÉES RENVOYANT DES VALEURS MULTIPLES	15
3 - REQUÊTES IMBRIQUÉES MULTIPLES	15
E - REQUETES D'INSERTION	16
F - LES REQUETES DE MODIFICATION	17
G - LES REQUETES DE SUPPRESSION	18
H) METHODE POUR ECRIRE UNE REQUETE	19
ANNEXE 1 : LES TABLES DE L'EXEMPLE	21

A - GENERALITES

Qu'est-ce que SQL ?

L'utilisation du langage SQL (Structured Query Language) permet de normaliser le développement des applications de bases de données relationnelles dans les entreprises.

SQL est un langage évolué de manipulation de bases de données relationnelles, il opère sur des ensembles logiques de données appelés relations.

SQL comprend un ensemble de commandes permettant de définir, de mettre à jour et d'afficher les informations des tables.

SQL a été développé chez IBM dans le milieu des années 70. Adopté dès sa parution par de nombreuses sociétés, il est considéré aujourd'hui comme une norme dans les environnements de grandes et moyennes entreprises.

Note pour les exercices et la connexion à la base d'exemples :

Démarrer – Programme – SQL anywhere 5.0 – Sybase Central – Tools connect

USER ID : dba

PASSWORD sql

Database file C:\bdd\exo_sql.db

Cliquez sur exo-SQL

Cliquez sur exo-SQL(dba)

FILES – OPEN ISQL

Saisir les requêtes SQL dans la fenêtre commande.

B - REQUETES DE SELECTION SUR UNE TABLE

1 - L'instruction SELECT

```
SELECT nom champ1, nom champ2, ...
FROM table ;
```

Le nom de champ sert d'en tête de colonne, pour utiliser un autre titre, appelé alias, utiliser l'instruction AS

Ex : Liste des employés et date de naissance

```
SELECT nom, date_naissance as «date de naissance» FROM employes
```

Remarque : SELECT * FROM table;

Cette instruction permet d'afficher toutes les colonnes.

Exercices :

1 - Liste de toutes les informations des employés

2 - Liste des employés (nom, prénom)

3 - Liste des clients (société, contact))

2 - Clause DISTINCT

N'affiche qu'un exemplaire de chaque jeu de lignes identiques.

```
SELECT DISTINCT nom champ1, nom champ2, ...
FROM table ;
```

Ex : Liste des produits achetés

```
SELECT DISTINCT ref_produit FROM Details_commandes
```

3 - Clause WHERE

L'instruction WHERE permet d'introduire une condition pour spécifier les lignes à afficher.

```
SELECT nom champ1, nom champ2, ...
FROM table
WHERE condition
```

Ex : Liste des clients dont la fonction est chef des ventes

```
SELECT code_client, societe
FROM clients
WHERE fonction = 'chef des ventes'
```

Remarque :

Pour construire une condition on dispose de plusieurs opérateurs : = , >= , <= , > , < , <> et ! qui est la négation soit : != , > , < , <> et !=

Pour combiner plusieurs conditions, utiliser les opérateurs logiques AND, OR et NOT.

Pour comparer un champ à du texte, on utilisera le caractère " " pour encadrer le texte. Pour comparer un champ à une valeur numérique, ce caractère ne doit pas être employé.

Exercices :

- 4 - Liste des clients habitant London
- 4bis- Liste des clients habitant Marseille et Nantes
- 5 - dates des commandes réalisées par l'employé 8
- 6 - Date d'embauche de l'employé nommé King
- 7 - Quantité en stock et libellé du produit réf 39
- 7bis- Employés embauchés après le 25 janvier 93

4 - Utilisation des attributs Between, In, Like

Ces instructions permettent de spécifier les conditions de recherche.

- **Between** peut-être utilisé pour vérifier si une valeur de colonne (caractère, date, numérique) se situe entre 2 valeurs.

Ex : Liste des produits dont la quantité en stock est comprise entre 100 et 200

```
SELECT ref_produit, unites_en_stock
FROM produits
WHERE unites_en_stock BETWEEN 100 and 200
```

- **In** vérifie si une valeur de colonne correspond à l'une de celle spécifié dans la liste
- **Not In** vérifie si une valeur de colonne ne correspond pas à l'une de celle spécifié dans la liste

Ex : Liste des clients n'habitant pas à Paris, Montréal ou Londres

```
SELECT societe, ville
FROM clients
WHERE ville NOT IN ('Paris', 'Montreal','London')
```

- **Like** sélectionne les lignes en comparant la valeur d'une colonne de type caractère à une chaîne de caractères spécifiée.

Remarque : Le caractère % représente n'importe quel nombre de caractères.
Le caractère _ représente un seul caractère quelconque.

Ex : Liste des clients du département 75

```
SELECT societe, code_postal
FROM clients
WHERE code_postal like '75%'
```

Ex : Liste des employés dont la fonction contient le mot " assistant "

```
SELECT *
FROM employés
WHERE fonction like '%assistant%'
```

Exercice :

8 – Nom de société, contact du client habitant une adresse dont le nom contient le mot " place "

5 - Utilisation d'expressions

Dans la clause SELECT on peut :

- insérer des instructions descriptives qui seraient répétées à chaque ligne,
- définir une nouvelle colonne (colonne calculée)

```
|| SELECT nom champ1, nom champ2, ..., expression calculée, "expression répétée" ||
|| FROM table ; ||
```

Ex d'expression numérique :

Liste des lignes commandes avec le total facturé par ligne (hors remise)

```
SELECT ref_produit, quantite, prix_unitaire, quantite * prix_unitaire as total
FROM details_commandes
```

Liste des lignes commandes dont le montant est inférieur à 100 f

```
SELECT n_commande, prix_unitaire * quantite as montant
FROM details_commandes
WHERE prix_unitaire * quantite <100
```

Ex d'expression sur les dates :

Liste des commandes avec le nombre de jour entre la date de livraison et la date de commande

```
SELECT n_commande, (a_livrer_avant)-(date_commande)
FROM commandes
```

Ex d'expression sur les caractères :

Afficher sur une seule colonne le nom et prenom des employés

```
SELECT nom + ' ' + prenom
FROM employes
```

Exercices :

9 - Nom du produit et évaluation du stock des produits référencés 38 et 39

6 - Utilisation de fonctions

Il existe 3 principaux types de fonctions dans le langage SQL : les fonctions mathématiques et statistiques, les fonctions de date et les fonctions relatives aux caractères.

a) Fonctions mathématiques et statistiques

COUNT () compte le nombre des lignes

SUM () additionne les valeurs d'une colonne numérique

MIN () trouve la valeur minimale d'une colonne de type caractère ou numérique

MAX () trouve la valeur maximale d'une colonne de type caractère ou numérique

AVG () calcule la moyenne des valeurs d'une colonne

Ex : Nombre d'employés

```
SELECT COUNT (*) FROM employes ;
```

Ex : Prix unitaire maximum, minimum et moyen

```
SELECT MAX(prix_unitaire), MIN(prix_unitaire), AVG(prix_unitaire)
FROM produits
```

Exercices :

10 - Prix unitaire du produit le plus cher

10 bis – Délai moyen, maximum et minimum entre commande et livraison

11 - Nombre de commandes effectuées par le client codé LILAS

11 bis – Date de naissance de l'employé le plus jeune

b) Fonctions de date

Current Date : renvoie la date du jour

Month() : renvoie le mois d'une colonne de type date

Year() : renvoie l'année d'une colonne de type date

Day() : renvoie le jour d'une colonne de type date

Ex : Liste des employés nés au mois de janvier

```
SELECT *
FROM employes
WHERE month(date_naissance)=1
```

Exercices :

12 - Liste des employés embauchés en 1992

13 - Liste des employés embauchés en 1993 et 1994

c) Fonctions relatives aux caractères

LEFT(colonne, i) : renvoie les i caractères situés à gauche de la colonne

RIGHT(colonne, i) : renvoie les i caractères situés à droite de la colonne

LENGTH(colonne) : renvoie le nombre de caractères d'une colonne

SUBSTR(colonne, i, j) : renvoie les j caractères de la colonne à partir de la ième position

Ex : Liste des produits dont la référence se termine par 5

```
SELECT ref_produit
FROM produits
WHERE right(ref_produit,1)='5'
```

Ex : Nombre de clients dans le département 67
 SELECT COUNT (*)
 FROM clients
 WHERE LEFT(Code_postal,2)='67'

7 - Classement des données

La clause ORDER BY spécifie la liste des colonnes sur lesquelles l'affichage doit être trié (ASC ou DESC).

Ex : Liste des employés classés selon la date d'embauche
 SELECT nom, date_embauche
 FROM employes
 ORDER BY date_embauche

Exercice :

14 - Classer par ordre décroissant de prix, les produits vendus dans l'entreprise

8 - Regroupement d'enregistrements

a) - Clause GROUP BY

Cette clause regroupe les lignes ayant des valeurs identiques dans une ou plusieurs colonnes issues de l'instruction SELECT, en une ligne.

Remarque : Chaque colonne figurant dans la clause select doit figurer dans la clause Group By.

Ex : Nombre d'articles différents par fournisseur
 SELECT n_fournisseur, count(*)
 FROM produits
 GROUP BY n_fournisseur

Ex : Nombre d'employés par fonction
 SELECT fonction, count (*)
 FROM employes
 GROUP BY fonction

Exercices :

15 - Quantités commandées par produit

16 - Nombre de produits commandés par commande

17 - Nombre de fois où chaque produit est commandé

b) - Clause Having

Cette clause permet de restreindre la sélection des groupes figurant dans le résultat. Elle précise la condition à laquelle chaque groupe doit répondre.

Ex : Nombre d'articles par commande, pour les n° de commandes après 10030

```
SELECT n_commande, count(*) as «Nombre d'articles»  
FROM details_commandes  
GROUP BY n_commande  
HAVING n_commande > 10030
```

Exercices :

18 - Nombre de produits commandés dans la commande 10250

19 - Liste des codes produits commandés au moins 50 fois

C - REQUETE DE SELECTION MULTITABLES - LES JOINTURES

Une jointure réalise une liaison entre deux tables en se basant sur l'égalité entre l'un des attributs de chaque table.

Deux solutions sont possibles pour écrire une jointure interne.

Citer les attributs recherchés dans la clause SELECT et préciser dans la clause WHERE la condition d'égalité entre les attributs qui réalisent la jointure entre les tables. La clause FROM doit contenir le nom de toutes les tables. La clause WHERE permet toujours de définir les critères de sélection combiné avec la condition de jointure par un opérateur AND.

On peut également définir la jointure en précisant les attributs de liaison entre les tables par la clause JOIN....ON.... Cette clause permet de séparer la condition de jointure et les conditions additionnelles de filtres.

Remarque : le produit cartésien de deux tables est équivalent à une jointure sans la condition de jointure.

Explication de la jointure

Exemple : Liste des ventes par employé

Commandes

N_commande	Code_client	N_employe	...
01	A1	100	
02	A12	100	
03	A7	100	
04	A10	101	

Employes

N_employe	Nom	Prenom	...
100	Dupont	Pierre	
101	Martin	Paul	
102	Dubois	Jack	

Après jointure le résultat donnera

N_commande	Code_client	N_employe	Nom	Prenom	...
01	A1	100	Dupont	Pierre	
02	A12	100	Dupont	Pierre	
03	A7	100	Dupont	Pierre	
04	A10	101	Martin	Paul	

1 - Jointure de 2 tables

Ex : Liste des ventes des employés

```
SELECT employes.nom, commandes.n_commande
FROM employes,commandes
WHERE employes.n_employe=commandes.n_employe
```

Autre écriture :

```
SELECT employes.nom, commandes.n_commande
FROM employes JOIN commandes ON
employes.n_employe=commandes.n_employe;
```

Ex : Vente du vendeur Fuller

```
SELECT commandes.n_commande, employes.Nom
FROM employes,commandes
WHERE employes.n_employe = commandes.n_employe
AND nom = 'Fuller';
```

Autre écriture :

```
SELECT commandes.n_commande, employes.Nom
FROM employes JOIN commandes ON
employes.n_employe = commandes.n_employe
WHERE nom = 'Fuller'
```

Ex : Liste des commandes avec dates et vendeurs

```
SELECT n_commande, date_commande, Nom
FROM commandes, employes
WHERE employes.n_employe = commandes.n_employe;
```

Autre écriture :

```
SELECT n_commande, date_commande, Nom
FROM commandes JOIN employes ON
employes.n_employe = commandes.n_employe
```

Ex : Nombre de ventes de chaque vendeur

```
SELECT employes.n_employe, nom, COUNT(*)
FROM employes, commandes
WHERE employes.n_employe = commandes.n_employe
GROUP BY employes.n_employe,nom
```

Autre écriture :

```
SELECT employes.n_employe, nom, COUNT(*)
FROM employes JOIN commandes ON
employes.n_employe = commandes.n_employe
GROUP BY employes.n_employe,nom
```

Exercices :

19 bis - Numéro et nom des employés ayant pris plus de 10 commandes en France

20 - Libellé des sociétés françaises ayant commandé avant le 31/07/1993

21 - Liste des commandes et date des commandes du client codé BONAP

22 - Calendrier des ventes de l'employé Fuller

22 bis - Nombre de commandes par pays selon la nationalité du client

22 ter - Liste et nombre de commandes des clients français ayant réalisé plus de 10 commandes en 1993 ou plus tard

- Jointure de plus de 2 tables

La jointure fonctionne selon le même principe qu'avec 2 tables, il faut juste procéder à plusieurs jointures

Ex : Liste des commandes avec le numéro de commande, le nom de l'employé et le nom du client

```
SELECT employes.Nom, n_commande, Societe
FROM employes, commandes, clients
WHERE employes.n_employe = commandes.n_employe
AND commandes.code_client=clients.code_client
ORDER BY employes.nom
```

Autre écriture :

```
SELECT employes.Nom, n_commande, Societe
FROM commandes
JOIN employes ON employes.n_employe = commandes.n_employe
JOIN clients ON commandes.code_client=clients.code_client
ORDER BY employes.nom
```

Exercices :

23 - Quantités du produit référencé 38, pour chaque commande de ce produit, vendu après le 31/07/93

23 bis - Liste des produits de référence 20 à 38 vendus à plus de 100 exemplaires après le 19-07-1993.

On affichera le nom du produit, la référence et la quantité.

24 - Détails de la commande 10250 (libellé produit, quantité, prix unitaire)

25 - Libellé des produits vendus par Fuller

26 - Villes dans lesquelles habitent les clients de Fuller

27 - Chiffre d'affaire de Fuller

27 bis - Liste des employés ayant un chiffre d'affaire > 1000 avec des clients habitant en France.

(on affichera :nom de l'employé et Chiffre d'affaire)

3 - Jointure d'une table avec elle-même (auto-jointure)

On peut joindre une table avec une copie de celle-ci. Attention, Il faut définir différents noms ou alias pour la table et sa copie.

Ex : Liste des employés et de leur chef de service classés par ordre décroissant des noms de chefs de service

```
SELECT Super.nom, 'est le chef de', Emp.nom
FROM employes Emp, employes Super
WHERE Super.n_employe= Emp.rend_compte_a
ORDER BY Super.nom DESC
```

Emp et Super sont les alias de la table employés.

Autre écriture :
SELECT Super.nom, 'est le chef de', Emp.nom
FROM employes Emp JOIN employes Super ON
Super.n_employe= Emp.rend_compte_a
ORDER BY Super.nom DESC

Exercices :

28 - Liste des vendeurs embauchés avant leur supérieur

4 - Jointure externe

Les jointures précédentes sont qualifiées de jointures internes, si la condition de jointure n'est pas vérifiée, l'enregistrement n'apparaît pas dans le résultat. Pour visualiser tous les enregistrements, y compris ceux ne répondant pas au critère, il faut réaliser une jointure externe

Ex : Liste des clients qui n'ont rien acheté

```
SELECT DISTINCT Societe, N_commande
FROM Clients
LEFT OUTER JOIN Commandes ON
Clients.Code_client = Commandes.Code_client
WHERE N_commande is null
```

D - REQUETES IMBRIQUEES

On peut employer des requêtes imbriquées pour joindre deux requêtes.

La construction de la commande SELECT supérieure dépend du nombre de valeurs renvoyées par la requête imbriquées.

1 - Requêtes imbriquées renvoyant une valeur unique

Ex : Ensemble des commandes du client nommé « BON APP »

```
SELECT n_commande,date_commande
FROM commandes
WHERE code_client =
(SELECT code_client
FROM clients
WHERE societe = 'Bon app')
ORDER BY date_commande
```

Ex : Ensemble des commandes du client nommé "BON APP" ou par le vendeur " FULLER ".

```
SELECT n_commande,date_commande
FROM commandes
WHERE code_client =
(SELECT code_client
FROM clients
WHERE societe = 'Bon app')
OR n_employe =
(SELECT n_employe FROM employes WHERE nom='Fuller') ;
```

Exercice :

29 - Liste des vendeurs embauchés avant King

2 - Requêtes imbriquées renvoyant des valeurs multiples

Ex : Liste des commandes et clients du département 44

```
SELECT n_commande, code_client, date_commande
FROM commandes
WHERE code_client IN
(SELECT code_client FROM clients WHERE code_postal like '44%')
ORDER BY n_commande
```

3 - Requêtes imbriquées multiples

Une sous requête peut contenir d'autres sous requêtes imbriquées

Ex : Ensemble des clients ayant commandé le produit "50"

```
SELECT code_client, societe, contact, ville
FROM clients
WHERE code_Client IN
(SELECT code_client FROM commandes WHERE n_Commande IN
(SELECT n_Commande FROM details_commandes WHERE ref_produit='50')) ;
```

E - REQUETES D'INSERTION

SQL Permet également d'insérer des données dans un table.

Ex : Ajouter le client : société « la nouvelle » de code « LANOUV »

```
INSERT INTO (code_client, societe)  
VALUES ('lanouv', 'la nouvelle')
```

Attention, ici tous les champs n'ont pas été renseignés.

Cela n'empêche pas la requête de fonctionner mais certaines données peuvent être indispensable au bon fonctionnement de la table et ne doivent pas être oubliées.

C'est en particulier le cas des identifiants.

F - LES REQUETES DE MODIFICATION

Syntaxe : UPDATE liste_table SET colonne1 = {expression/instruction_select},
colonne2 = {expression/instruction_select}
FROM liste_tables
WHERE condition
ORDER BY expression [ASC | DESC]

Si on ne spécifie aucune condition dans la clause WHERE, l'update sera réalisé sur toutes les lignes de la table.

Normalement, l'ordre de mise à jour des lignes importe peu. Cependant, il existe certains cas où on peut avoir intérêt à spécifier cet ordre (ex : si on veut augmenter un champ clé de 1, il faut commencer par la fin des enregistrements). On utilisera dans ces cas la clause ORDER BY.

Ex : UPDATE produits SET prix_unitaire = prix_unitaire * 1.12 WHERE code_categorie = '1'

Ex : UPDATE produits p SET prix_unitaire = prix_unitaire * 2 from produits p, categories c
WHERE c.code_categorie = p.code_categorie

Exercice :

Augmenter le prix des produits de la catégorie « desserts » de 10%

G – LES REQUETES DE SUPPRESSION

Syntaxe : DELETE [FROM] nom_table
... [FROM table-list]
... [WHERE search-condition]

Le 1^{er} FROM est optionnel avec Sybase ASE. Il est toutefois conseillé de l'utiliser pour conserver la compatibilité avec SQL2.

Le 2^{ème} FROM est optionnel et indique les tables impliquées dans la jointure par la condition WHERE.

Attention : si aucune condition WHERE n'est appliquée, toutes les lignes de la table seront supprimées.

Ex : suppression des clients habitant à Paris

DELETE from client WHERE ville = 'Paris'

Exercice :

Supprimer les commandes passées par l'employé 'dodsworth'

H) METHODE POUR ECRIRE UNE REQUETE

Préalable : bien connaître le modèle et comprendre précisément la question.

1°) Etablir la liste des informations à traiter :

On distinguera :

- Les informations à afficher
 - o Champs à afficher
 - o Calculs simples sur x champs (ex : qté * prix_unitaire)
 - o Agrégation d'un champ => fonctions d'agrégation comme COUNT, SUM, MIN, MAX
 - ...

Dans le cas d'une agrégation, soit on veut obtenir un seul résultat (dans ce cas, on ne peut pas faire apparaître d'autres champs dans le select.) soit on veut obtenir x lignes de résultat (ex : le nombre de produits commandés par commande) : dans ce cas, il faut obligatoirement créer un niveau de regroupement : GROUP BY.

- Les informations de restriction de sélection. On en a de 2 types :
 - o Des informations de restriction qui portent sur des lignes d'une table => WHERE
 - o Des informations de restrictions d'affichage des groupes (celles-ci portent en général sur des fonctions d'agrégation) => HAVING

2°) Ecrire le SELECT (cf. partie « informations à afficher)

3°) Ecrire le FROM

La liste des tables se définit à partir de :

- des champs à afficher (cf. SELECT)
- des champs de restriction sur les lignes (WHERE)
- des champs de restriction sur les groupes (HAVING)
- des jointures nécessaires pour relier les tables (jointure classique ou jointure gauche ou droite)

4°) Ecrire le WHERE

On distinguera ici :

- les conditions de restriction sur les lignes
- les conditions de jointure

Dès qu'on réalise une jointure : on s'interrogera sur le type de jointure (normale, gauche ou droite) et on s'assurera de la pertinence du résultat obtenu (on obtient toujours le nombre de lignes qui correspond au plus petit niveau de détails).

5°) Ecrire le GROUP BY

Quand on crée un groupe, il faut s'assurer de la cohérence du groupe créé avec le niveau d'affichage souhaité.

Quand j'ai un GROUP BY, je ne peux faire apparaître dans le SELECT que :

- des champs qui sont repris dans le GROUP BY
- des fonctions d'agrégation (SUM, MIN, MAX ...)

6°) Restreindre l'affichage des groupes => HAVING

On écrira ici la condition de restriction portant sur les groupes. En général, celle-ci porte sur la fonction d'agrégation. (les restrictions concernant les champs s'écrivent dans la clause WHERE).

7°) Trier les enregistrements (ORDER BY)

Cette méthode s'applique pour les requêtes de sélection (type SELECT). Pour les autres types de requêtes, le principe reste le même.

ANNEXE 1 : Les tables de l'exemple

Tables :

Clients
 Commandes
 Employes
 Messagers
 Details_commandes
 Produits
 Fournisseurs
 Categories

